ABBA

NEW MEMBER STARTER KIT HANDBOOK

Everything a new team member needs to understand how the pieces fit together.

MAY 28, 2026

SILPA LABS
A Division of Silpa Consulting LLC

What ABBA is (and is not)

Welcome to the team. Before you touch any code, it helps to know what you are building and why it is shaped the way it is. This chapter gives you that picture in plain language.

The product in one paragraph

ABBA is a clinic operations and coordination platform for clinics that deliver ABA (Applied Behavior Analysis) therapy. It coordinates the whole journey: intake, matching a client to the right clinician, scheduling, communication, documentation, and management oversight.

Important: ABBA is not an electronic health record (EHR), and it is not a billing engine. It *integrates with* those systems, but its job is coordination. Keep that boundary in mind; it explains a lot of the design decisions you will see later.

Who uses it (the people / roles)

ABBA serves several kinds of people. These are the same names used in the code, the database, and the planning documents, so learn them once and they will keep showing up:

- Admin: a system administrator who manages users, roles, and platform configuration; the highest-privilege role.
- Clinician: a practitioner who delivers care. Each clinician has an expertise profile.
- Supervisor: a clinician who oversees others (supervisees) and can review their expertise and work.
- Coordinator: staff who manage intake review, matching decisions, and scheduling.
- Leader / Leadership: clinic management; they see utilization and analytics. Analytics are explicitly non-punitive.
- Customer: the person or family receiving services. A customer record is created only from an approved intake.
- Family user: a customer's family member who gets scoped portal access to *their connected customers only*.

The seven RBAC roles

Access in ABBA is governed by RBAC (Role-Based Access Control), enforced in the backend using a RolesGuard together with a @Roles() decorator. The full set of roles used across the build is:

Admin · Leader · Supervisor · Clinician · Coordinator · Customer · Family

The exact permissions for each role live in the permission matrix (in docs/, task ABBA-53). The key idea to internalise now: the role decides what each person is allowed to do, and that decision is always made on the server.

How the pieces fit together

This is the mental model the whole team shares. If you understand these five ideas, most of the codebase will make sense.

The five core ideas

1. The frontend never talks to the database. The web app calls the API through a typed client, and both sides validate data with shared Zod schemas so the client and server always agree on the shape of things.
2. The backend is the gate. RBAC, audit logging, and consent are all enforced in the backend. The UI only *reflects* permissions; it is never the thing that enforces them.
3. Slow work runs on the worker. Long or slow jobs (background tasks, AI calls) run on a separate process called api-worker, never inline inside a web request, so jobs can never starve web traffic.
4. Providers degrade gracefully. Telehealth, EHR, and AI sit behind provider interfaces with NoOp fallbacks. If the hardware or credentials are missing, the feature quietly turns off instead of crashing.
5. State machines live in the backend. Intake and documentation move through fixed, server-enforced states. The client never invents its own transition.

The runtime, drawn out

Everything runs as Docker containers on a single on-premises server, sitting behind nginx. Here is what each container does:

Container	Role	Notes
nginx	Reverse proxy + TLS termination	Routes / to web and /api to the API
web	Next.js frontend	Served behind nginx
api	NestJS API (web requests only)	Never runs background jobs
api-worker	NestJS worker (pg-boss jobs, AI calls)	Kept separate so jobs never starve web requests
postgres	PostgreSQL database	The source of truth, and the pg-boss job queue
ollama	Local AI model runtime	Optional; AI degrades to NoOp if absent
fhir	HAPI FHIR sandbox (dev/test EHR)	Read-only EHR source

Source control and CI/CD run on GitHub. CI (the checks on your pull request) runs on GitHub-hosted runners. CD (the deploy) runs on a self-hosted runner installed on the server itself: GitHub never reaches into the server; the runner reaches out to GitHub. No inbound ports are opened just for deployment.

The shared vocabulary

The team, the product requirements, the Jira tasks, and the code all use the same words on purpose. When a new domain term or system concept enters the project, it gets added here. Skim this chapter now, then come back to it whenever a word trips you up.

Core data entities

These are the main things ABBA stores. The phase in brackets tells you roughly when each one shows up in the build:

- User / Role / UserRole / Session: identity and access *(Phase 1)*.
- Audit log: an append-only record of privileged actions *(Phase 1 onward)*.
- Consent record: per-subject, per-channel consent state, with history *(Phase 2)*.
- Clinician profile / expertise: the five dimensions are populations, modalities/approaches, languages, training/experience, and constraints (location, telehealth, hours) *(Phase 2)*.
- Intake submission: a prospective customer's request, which moves through a state machine *(Phase 2)*.
- Customer profile: created only by approving an intake, in a single transaction *(Phase 2)*.
- File: an uploaded document; its metadata lives in the database, the bytes live on a disk volume *(Phase 2)*.
- Match run / recommendation / factor / decision: the deterministic matching output *(Phase 3)*.
- Appointment: a scheduled session with a state and optional recurrence *(Phase 3)*.
- Reminder: a scheduled, email-first nudge with confirm/cancel tokens *(Phase 3)*.
- Family connection: links a family user to one or more customers *(Phase 4)*.
- Documentation: session notes moving through a review workflow *(Phase 5)*.
- Billing record / insurance status: integration and tracking data *(Phase 5)*.
- Compliance rule / check: policy evaluation plus reporting *(Phase 5)*.

The state machines (server-enforced)

Three workflows move through fixed states. The server enforces every transition; the client can request one, but it never decides one.

Workflow	States
Intake	started → submitted → reviewed → approved (plus → rejected, which requires a reason). Approval creates the customer profile transactionally.
Appointment	scheduled → confirmed → completed, plus canceled and no-show.
Documentation	draft → submitted → reviewed.

System & architecture terms

- Monorepo: one repository holding multiple apps and packages, wired together with pnpm workspaces.

- 'api' vs 'api-worker': the web API process versus the separate process that runs background jobs and AI calls, so jobs never starve web requests.
- Provider interface + NoOp fallback: a swappable seam (AIProvider, NotificationProvider, TelehealthProvider, EHRProvider) with a NoOp default, so features degrade gracefully when hardware or credentials are absent.
- pg-boss: a Postgres-backed job queue (so no extra Redis is needed), processed by api-worker.
- RBAC: Role-Based Access Control, enforced in the backend.
- Audit log (append-only): an immutable trail of privileged actions.
- Consent gate: a check run before every outbound message; a blocked send is logged, not delivered.
- Deterministic matching: a weighted scoring service (no LLM). Explanations are templates rendered from the score factors, so matching is fast, free, and auditable.
- Advisory AI: all AI output (intake summaries, doc drafts) is labelled advisory, logged, and human-reviewed, never autonomous. It runs locally via Ollama (a small model) behind the AIProvider seam.
- FHIR: the healthcare data standard ABBA reads EHR data through (against a local HAPI FHIR sandbox in dev). Reads are read-only and carry provenance labels.
- Self-hosted runner: a GitHub Actions runner on the clinic server that performs deploys; GitHub never reaches into the server.
- Definition of Done (DoD): the shared bar a task must clear before it is "done" (see Chapter 6).

Planning terms

- ABBA-NN: a Jira task ID (ABBA-1 through ABBA-160), each mapped to one developer in the per-developer backlogs.
- Epic / Phase: work is grouped into Phases 0 through 5; each Phase is a Jira Epic.
- Critical path: the longest dependency chain in a phase, which determines how long that phase takes (see ABBA_Coordination_Map.md).
- MVP: reached when the Phase 3 e2e suite (ABBA-110) is green, meaning intake → match → schedule → remind → confirm works end to end.

Getting your machine running

This chapter gets the ABBA platform running locally so you can start contributing. If anything here turns out to be wrong or missing, fix it in the same pull request; keeping this accurate is everyone's job.

Prerequisites

- Node.js (LTS) and pnpm: the repo is a pnpm workspace monorepo.
- Docker Engine plus the Docker Compose plugin.
- Git, with access to the repo.
- Around 8 GB of free RAM is comfortable (Postgres + API + worker + web + optional Ollama).

First-time setup

Run these steps once, in order:

1. Clone

git clone <repo-url> abba && cd abba

2. Install all workspace dependencies

pnpm install

3. Create your local env file from the template and fill in dev values

cp .env.example .env

(the defaults in .env.example are wired for the local Docker stack)

4. Bring up the full stack

docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d

5. Apply database migrations

docker compose exec api pnpm prisma migrate deploy

6. Seed demo data (idempotent, safe to re-run)

docker compose exec api pnpm seed

Then open:

- Web: http://localhost (through nginx), or the dev port in your compose override.
- API health: http://localhost/api/health should return 200.

Heads up: The seed script prints default logins (one admin plus sample users). Never use seed credentials anywhere but local or demo environments.

Day-to-day commands

- pnpm dev # run apps in watch mode (per workspace scripts)
- pnpm lint # repo-wide lint (one shared ruleset)
- pnpm test # unit tests
- pnpm build # build all apps/packages
- docker compose logs -f api # tail API logs (Pino JSON)
- docker compose logs -f api-worker # tail the job worker
- docker compose exec api pnpm prisma studio # inspect the DB

When you change the Prisma schema:

docker compose exec api pnpm prisma migrate dev --name <short-description>

commit the generated migration file: schema changes ALWAYS ship as a migration

Where everything lives (repository layout)

apps/

web/ Next.js frontend (App Router): (public)/(protected) route groups
api/ NestJS API: modules/, common/, config/

packages/

ui/ Shared component library (Tailwind + shadcn/ui) [D3 + D4 co-own]
shared/ Shared TS types, enums, Zod schemas (the one contract layer)
config/ Shared ESLint / Prettier / tsconfig presets

infra/ Dockerfiles, compose, nginx [D5 owns]

docs/ This runbook, contracts, glossary, ADRs, standards

scripts/ Root scripts

.github/ CI/CD workflows, PR/issue templates [D5 owns]

CODEOWNERS routes infra/ and .github/ to Developer 5: pipeline and infra changes need their review.

Where to look when you are stuck

Your question	Where to look
What am I building, and in what order?	Your per-developer Jira backlog, plus ABBA_Coordination_Map.md
What does this domain term mean?	Chapter 3 here, or docs/glossary.md
What is the API shape?	Chapter 7 here, or docs/api-contracts.md
How do we deploy or recover?	Chapter 8 here, or docs/runbook.md
What are the engineering rules?	Chapters 5 and 6 here, CONTRIBUTING.md, and 07: DevOps Standards

Environment variables

ABBA is configured through environment variables. You copy .env.example to .env and fill in real values. Never commit a real '.env'. The defaults in the template are wired for the local Docker stack, so a fresh clone mostly just works.

Two rules that protect you: Every variable is validated at API startup: a missing or invalid one should fail fast with a clear error naming the offending variable. And every time you introduce a new variable, you must add it to .env.example with a comment; that is part of the Definition of Done. Finalising validation and the full list is task ABBA-18 (Developer 1).

The variables, grouped

Below is the starter template, grouped the way the file is. The owner in each heading tells you who looks after that area.

Core

NODE_ENV=development # development | production | test

LOG_LEVEL=info # pino level: trace|debug|info|warn|error

API_PORT=3001 # NestJS API port (behind nginx /api)

APP_BASE_URL=http://localhost # public base URL (email links, token URLs)

Database (PostgreSQL): used by api, api-worker, and pg-boss

POSTGRES_USER=abba_app # least-privilege app user (NOT a superuser)

POSTGRES_PASSWORD=change-me-locally # real value only in the server .env

POSTGRES_DB=abba

POSTGRES_HOST=postgres # docker service name

POSTGRES_PORT=5432

DATABASE_URL=postgresql://abba_app:change-me-locally@postgres:5432/abba?schema=public

Auth / sessions (Developer 1)

JWT_ACCESS_SECRET=change-me-32+chars-random # rotate per runbook (Chapter 8)

JWT_REFRESH_SECRET=change-me-32+chars-random

JWT_ACCESS_TTL=15m

JWT_REFRESH_TTL=7d

COOKIE_DOMAIN=localhost

COOKIE_SECURE=false # true in production (HTTPS only)

Email / notifications (Developer 5, Nodemailer to SMTP relay)

NOTIFICATIONS_EMAIL_ENABLED=true

SMTP_HOST=localhost

SMTP_PORT=1025 # e.g. a local mailpit/mailhog catcher in dev

SMTP_USER=

SMTP_PASSWORD=

SMTP_FROM="ABBA <no-reply@abba.local>"

Background jobs, file storage

```
# pg-boss runs inside Postgres; processed by api-worker
PGBOSS_SCHEMA=pgboss
# File storage (Developer 2, local filesystem volume; bytes never in the DB)
FILE_STORAGE_PATH=/data/uploads
FILE_MAX_BYTES=10485760    # 10 MB upload cap (tune as needed)
```

Providers: AI, telehealth, EHR

Each of these defaults to noop, which is exactly why features degrade gracefully when a provider is not configured.

```
# AI provider (Developer 1 scaffold / Developer 5 Ollama), Phase 2 / 5
AI_PROVIDER=noop          # noop | local
OLLAMA_BASE_URL=http://ollama:11434   # used when AI_PROVIDER=local
OLLAMA_MODEL=llama3.2:3b    # small model for weak hardware
# Telehealth provider (Developer 5), Phase 4
TELEHEALTH_PROVIDER=noop    # noop | jitsi | zoom
JITSI_BASE_URL=https://meet.jit.si
ZOOM_ACCOUNT_ID=          # optional; only if TELEHEALTH_PROVIDER=zoom
ZOOM_CLIENT_ID=
ZOOM_CLIENT_SECRET=
# EHR provider (Developer 5), Phase 4 (read-only against FHIR sandbox)
EHR_PROVIDER=noop         # noop | fhir
FHIR_BASE_URL=http://fhir:8080/fhir   # local HAPI FHIR sandbox
```

Web (Next.js)

```
NEXT_PUBLIC_API_URL=/api    # frontend calls the API via nginx
```

How we work (contributing)

These are the engineering conventions every developer follows. This is the practical companion to 07: DevOps Standards, and it is the document each task's "Definition of Done" points to. (Authoring and refining it is task ABBA-20.)

Branching

- Branch off main. main is always deployable.

- Name branches type/short-description, for example feat/login-page, fix/audit-null-actor, chore/ci-cache.
- One logical change per branch. Keep PRs small enough to review in one sitting.

Commits

- Use present-tense, imperative summaries: “Add RolesGuard”, not “Added” or “Adds”.
- Conventional-style prefixes are encouraged: feat:, fix:, chore:, docs:, test:, refactor:.
- Never commit secrets, ‘.env’, or generated artifacts. CI's secret scan will fail the build if you do.

Pull requests

- Open against main. Fill in the PR template (what changed, how you tested, screenshots for UI).
- Link the Jira task (ABBA-xx) and its dependencies.
- At least one review approval is required. Branch protection blocks the merge until CI is green.
- Squash-merge to keep main history clean. Delete the branch after merge.
- Deploys to production are gated behind a human approval in the GitHub production environment (see Chapter 8).

Definition of Done (every task)

A task is done only when all of these are true:

- Merged to main via a reviewed PR.
- CI is green (lint, typecheck, test, build, and e2e from Phase 1 onward).
- It works in the local Docker stack (docker compose up), not just on your machine.
- Any new environment variable is added to .env.example (with a comment).
- Any schema change ships as a committed Prisma migration (never a hand-edit).
- New domain entities have soft-delete fields and audit coverage on privileged actions *(Phase 2 onward)*.
- Tests exist for new logic (*.spec.ts for modules, services, and guards).
- The acceptance criteria on the Jira task are all met.
- UI work includes screenshots in the PR.

Testing expectations

- Unit-test services, guards, interceptors, and pure logic.
- Use Supertest for API endpoints (happy path, plus permission denials and validation failures).
- Developer 5 owns the cross-cutting e2e suites (ABBA-35 / 78 / 110 / 133 / 159) that gate merges. Write your modules so they are testable from a fresh clone with seed data.

The standing rules

These are the rules that cause the worst bugs when ignored. Read them twice.

- Backend is the gate. RBAC, audit, and consent are enforced server-side; the UI only reflects them.
- One contract. Request/response shapes are Zod schemas in packages/shared, validated on both client and server. Don't define a shape twice.
- The frontend never touches the database. Everything goes through the API.
- State machines live in the backend. Intake and documentation transitions are server-enforced; the client never invents a transition.
- Slow work runs on the worker. Jobs and AI calls go through api-worker, never inline.
- Providers degrade gracefully. Telehealth, EHR, and AI sit behind interfaces with NoOp fallbacks.
- Audit is append-only. Never expose an update or delete path to audit_log.
- Freeze published contracts. Once an API contract is documented and others build on it, don't change the shape silently: coordinate.

Documentation

- Keep docs/api-contracts.md, docs/glossary.md, the runbook, and CONTRIBUTING.md current as part of the change that affects them, not “later”.
- Significant architecture choices get an ADR (Architecture Decision Record). The ADR folder and its template are part of task ABBA-20.

An ADR captures, for each significant decision: its status (proposed, accepted, or superseded), the date and deciders, the context (the situation and forces at play), the decision stated plainly, the consequences (what gets easier or harder, and the trade-offs), and the alternatives considered (what else was weighed, and why it was not chosen).

The API contracts

docs/api-contracts.md is the single source of truth for the API surface the frontend builds against. Backend owners append to it as endpoints land; the frontend confirms each section is sufficient before relying on it. Once a contract is published and others build on it, the team freezes the shape, changing it only by coordinating, never silently.

The skeleton is filled in across phases by these tasks: ABBA-31 (Phase 1 auth/users/roles), ABBA-60 (Phase 2 expertise/files), ABBA-91 (Phase 3 matching), ABBA-118 (Phase 4 telehealth/family/EHR), and ABBA-143 (Phase 5 analytics).

Conventions

- All endpoints are under /api. Auth is via httpOnly session cookies (no tokens in JS).
- Request/response bodies match the Zod schemas in ‘packages/shared’; link the schema name rather than re-describing the shape where possible.
- Standard error envelope: an HTTP status plus { "error": { "code", "message", "details?" } }.
- Role-gated endpoints note their allowed roles. RBAC is enforced server-side.
- Each endpoint is documented as: method + path, purpose, auth/roles, request, response, and errors.

Template for one endpoint

POST /api/<resource>

Purpose: <one line>

Auth: <public | authenticated> · Roles: <Admin, Coordinator, ...>

Request: <shared schema name> { ...key fields... }

Response 200: <shared schema name> { ... }

Errors: 400 validation · 401 unauth · 403 role · 404 not found · 409 conflict

Phase 1: Auth, Users, Roles

Owner: Developer 1 / Developer 2 · task ABBA-31.

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- POST /api/auth/refresh
- POST /api/auth/password-reset/request and POST /api/auth/password-reset/confirm
- POST /api/auth/verify-email
- GET /api/users/me and PATCH /api/users/me
- GET /api/users *(Admin)* and GET /api/users/:id *(Admin)*
- POST /api/roles/assign and POST /api/roles/revoke *(Admin)*

Phase 2: Expertise & Files

Owner: Developer 2 · task ABBA-60.

- GET /api/clinicians and GET /api/clinicians/:id
- PUT /api/clinicians/:id/expertise *(self / supervisor)*
- GET /api/clinicians/search: a filterable expertise query, a **FROZEN shape** that feeds matching.
- GET /api/taxonomies/{populations|modalities|languages}
- POST /api/files *(upload)* and GET /api/files/:id *(download, RBAC + ownership)*
- Intake & customers endpoints *(owner: Developer 4)*: POST/GET/PATCH /api/intake..., transitions, and GET/PATCH /api/customers/:id

Phase 3: Matching

Owner: Developer 2 · task ABBA-91.

- POST /api/matching/run: returns ranked recommendations plus explanations.
- POST /api/matching/:id/approve and POST /api/matching/:id/override *(reason required)*
- Scheduling *(owner: Developer 5)*: POST/PATCH /api/appointments..., GET /api/schedule/utilization
- Reminders *(owner: Developer 5)*: GET/PATCH /api/reminders/cadence; plus public GET /api/appointments/confirm?token= and /cancel?token= *(owner: Developer 1)*

Phase 4: Telehealth, Family, EHR

Owner: Developer 2 (docs) · task ABBA-118.

- Family *(owner: Developer 4)*: POST/GET/DELETE /api/family/connections, GET /api/family/appointments, POST /api/family/scheduling-requests
- Telehealth *(owner: Developer 5)*: GET /api/appointments/:id/telehealth-link
- EHR *(owner: Developer 5, read-only)*: GET /api/ehr/patient/:id, GET /api/ehr/appointments/:id: responses carry provenance metadata.

Phase 5: Analytics, Docs, Billing, Compliance, AI

Owner: Developer 2 (docs) · task ABBA-143.

- Analytics *(owner: Developer 2)*: GET /api/analytics/operations, GET /api/analytics/performance, GET /api/analytics/outcomes *(non-punitive)*
- Documentation *(owner: Developer 4)*: POST/GET/PATCH /api/documentation... *(state transitions)*
- AI *(owner: Developer 1)*: intake summary and doc draft are async, advisory, and human-reviewed.
- Billing / insurance *(owner: Developer 5)*: GET /api/billing/export, GET/PATCH /api/insurance/:customerId/status
- Compliance *(owner: Developer 1)*: GET /api/compliance/status, GET /api/compliance/report, GET /api/audit/export *(read-only)*

Operating the platform (runbook)

This is how the team deploys, recovers, and operates ABBA on the clinic's on-premises server. It is referenced by ABBA-6 (deploy/rollback), ABBA-19 (secret rotation), ABBA-77 (file restore), and ABBA-160 (backup-restore drill). The runbook is kept current on purpose: a runbook that is wrong is worse than none.

Audience: Primarily Developer 5 (DevOps) and whoever is on call. But everyone should be able to follow the deploy and rollback steps, which is why they are here.

Environments & access

- Local: each developer's machine via docker compose (plus dev overrides). See Chapter 4.
- Production: the on-prem server. Deploys are gated behind a GitHub Environment (production) that requires a human approval before the deploy job runs (see 07: DevOps Standards).
- Real secrets live only in the server's .env file (mode 600, owned by the deploy user). Never in Git.

First-time server setup (one-off)

1. Install Docker Engine and the Docker Compose plugin.
2. Create a least-privilege deploy user; add it to the docker group.
3. Clone the repo to the deploy path (for example /opt/abba).

4. Copy .env.example to .env; fill in real values (DB password, JWT secrets, SMTP, base URLs). Then chmod 600 .env.
5. Install the self-hosted GitHub Actions runner as a service under the deploy user, scoped to this repo only.
6. Provision TLS certs for nginx (Let's Encrypt if internet-reachable, otherwise an internal CA or self-signed for LAN-only).
7. First boot: docker compose up -d, then run migrations and seed.
8. Verify health, and that the approval-gated deploy.yml workflow can reach the runner.

Routine deployment

The normal, automated path:

1. A PR is reviewed and CI is green.
2. Squash-merge to main.
3. The deploy.yml workflow queues; approve it in the GitHub production environment.
4. The self-hosted runner pulls main, rebuilds images, runs docker compose up -d, runs pending migrations, then hits the health checks.
5. Confirm health, and watch the logs for about two minutes.

Manual deploy (only if the runner is down):

```
cd /opt/abba

git fetch origin && git checkout main && git pull

docker compose build

docker compose up -d

docker compose exec api pnpm prisma migrate deploy

# then run the health checks
```

Database operations

Migrations (Prisma, never hand-edit the schema in prod):

```
docker compose exec api pnpm prisma migrate deploy   # apply committed migrations

docker compose exec api pnpm prisma migrate status   # see what's applied/pending
```

Migrations are forward-only in production. To “undo” one, ship a new corrective migration.

Seed (idempotent, safe to re-run; used for demos and fresh installs):

```
docker compose exec api pnpm seed
```

Backup (nightly and automated; also run before risky changes):

```
docker compose exec -T postgres pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" \

 | gzip > /opt/abba/backups/db-$(date +%F-%H%M).sql.gz
```

Remember: Store backups off this server, encrypted. A backup you have never restored is not a backup.

File storage operations

Uploaded files live on a persistent Docker volume (not in the database), served only through the authenticated API (never as static assets). The examples use the volume name abba_uploads, mounted in the api/api-worker containers at the path set by FILE_STORAGE_PATH (default /data/uploads); adjust the name to match your docker-compose.yml if it differs.

Back up the uploads volume (run alongside the DB dump so they stay consistent):

```
docker run --rm -v abba_uploads:/data -v /opt/abba/backups:/backup alpine \
  tar czf /backup/uploads-$(date +%F-%H%M).tar.gz -C /data .
```

Restore files and database together (they must match, restore as a pair):

1. stop the app containers (keep postgres if restoring DB into it)

```
docker compose stop api api-worker web
```

2. restore the database

```
gunzip -c /opt/abba/backups/db-<stamp>.sql.gz \
  | docker compose exec -T postgres psql -U "$POSTGRES_USER" "$POSTGRES_DB"
```

3. restore the uploads volume

```
docker run --rm -v abba_uploads:/data -v /opt/abba/backups:/backup alpine \
  sh -c "rm -rf /data/* && tar xzf /backup/uploads-<stamp>.tar.gz -C /data"
```

4. bring the app back up and verify

```
docker compose up -d
```

Secrets & rotation (ABBA-19)

- Secrets live only in the server .env (mode 600). CI's secret-scanning step fails the build if a secret is committed.
- To rotate a secret (JWT signing key, DB password, SMTP password): generate the new value, then update .env on the server.
- For the JWT signing secret: rotating it invalidates existing sessions, so users re-login. Schedule it for low-traffic hours.
- For the DB password: change it in Postgres *and* .env in the same window, then restart api and api-worker.
- Run docker compose up -d to restart affected containers; then verify health.
- Never email or Slack a secret. Never paste one into a PR, an issue, or a log line.

Health checks & observability

- API health: curl -fsS https://<host>/api/health should return 200.
- Web: load https://<host>/ and complete a login.
- Worker: confirm api-worker is up (docker compose ps) and processing jobs (a queued reminder or AI job completes).
- Logs: structured JSON via Pino, with correlation IDs. Use docker compose logs -f api (or api-worker). Logs rotate by size and date.
- DB: docker compose exec postgres pg_isready.

Common incidents & first fixes

Symptom	Likely cause	First action
API won't boot, clear "missing env" error	Env validation failing (ABBA-18)	Check .env against .env.example; fix the named var
Web loads but every API call 500s	DB unreachable / migration pending	pg_isready; run migrate deploy; check DATABASE_URL
Jobs/reminders not sending	Worker down, SMTP misconfig, or consent blocking	Check worker logs; verify SMTP env; confirm it's not a correct consent drop
AI summaries say "unavailable"	Ollama down or absent	Expected graceful degradation: restart ollama or leave as NoOp
Telehealth "no link"	Provider = NoOp or Jitsi base URL unset	Set TELEHEALTH_PROVIDER + JITSI_BASE_URL
EHR view empty / erroring	FHIR sandbox down	Restart fhir; EHR reads are read-only and should degrade gracefully
Deploy stuck "waiting"	production approval gate	Approve the run in the GitHub Environment
Need to undo a bad release	N/A	Roll back (see below)

Provider note: Telehealth, EHR, and AI all sit behind interfaces with NoOp fallbacks. A missing or broken provider should degrade gracefully, not crash the app. If it crashes, that's a bug to file, not an emergency to hand-fix in prod.

Rollback & the backup-restore drill (ABBA-6, ABBA-160)

To roll back a release:

cd /opt/abba

git checkout <last-good-commit-or-tag>

docker compose build && docker compose up -d

If the bad release ran a migration, restore the DB from the pre-deploy backup.

Tag known-good releases so rollback targets are obvious.

The backup-restore drill (run at least once, and as part of ABBA-160):

1. Take a fresh DB and uploads backup.
2. On a scratch environment (or in a maintenance window), restore both as a pair.

Boot the stack; log in; open a record that includes an uploaded file; confirm the data and files came back together.

3. Document the date and result. If you have never completed this drill, your backups are unverified.

Standing operational rules (recap)

- Reviewed PR → CI green → squash-merge → approved deploy. No direct pushes to main.
- api and api-worker stay separate; AI and long jobs run on the worker only.
- Consent is checked before every outbound message; a blocked send is logged, not an error.
- Audit rows are append-only; never expose a mutable path to audit_log.
- Backups are encrypted, off-server, and tested.